

Firmware Sentinel-X — Sécurité

Le firmware du boîtier (NodeMCU ESP8266, v2.8.2) protège la chaîne capteur → serveur par deux mesures complémentaires : il s'authentifie auprès du broker MQTT et chiffre l'intégralité de son trafic en MQTTS (TLS 1.2), en vérifiant le certificat du serveur contre une autorité de certification (CA) épinglée dans le firmware.

Chiffrement du flux — MQTTS / TLS 1.2

Le boîtier ne communique avec le broker qu'en TLS, sur le port 8883. Le client réseau est un `BearSSL::WiFiClientSecure` configuré pour ne faire confiance qu'à la CA du groupe : le certificat présenté par le serveur est vérifié (signature par la CA épinglée et date de validité). Un serveur qui ne présenterait pas un certificat signé par cette CA est rejeté, ce qui bloque toute interception de type « man-in-the-middle ».

```
#if MQTT_USE_TLS
BearSSL::WiFiClientSecure net;          // connexion chiffrée, certificat
vérifié
BearSSL::X509List *caCert = nullptr;
#else
WiFiClient net;                        // mode clair (non utilisé en
production)
#endif

// --- setup() ---
caCert = new BearSSL::X509List(CA_CERT); // CA du groupe, épinglée dans le
firmware
net.setTrustAnchors(caCert);            // le boîtier ne fait confiance
qu'à elle

// --- à chaque connexion (mqttConnect) ---
net.setX509Time(time(nullptr));         // l'heure sert à valider la date
du certificat
```

Points clés :

- Port **8883 (MQTTS)** ; le mode clair 1883 est désactivé côté boîtier (`MQTT_USE_TLS 1`).
- `setTrustAnchors` = **épinglage** : seule la CA « Sentinel-X CA » est acceptée, aucune autre autorité publique.
- `setX509Time` : sans heure correcte, la date du certificat ne peut être vérifiée — le boîtier attend donc l'heure (NTP) avant de publier.

- Certificat serveur : clé **EC P-256** (prime256v1), signature **ECDSA-SHA256** — nettement plus léger que RSA pour l'ESP8266.

Authentification au broker

Le boîtier se connecte avec un identifiant et un mot de passe dédiés (compte `esp`). Le broker refuse toute connexion anonyme, donc un boîtier — ou un attaquant — sans identifiants valides ne peut ni publier ni s'abonner. Un couple user/mot de passe incorrect est rejeté avec l'état 5.

```
bool ok = mqtt.connect(DEVICE_HOSTNAME, MQTT_USER, MQTT_PASS,
                       topicStatus, 1, true, "offline", true);

if (!ok) {
    // mqtt.state() == 5 : identifiants refusés par le broker
    // mqtt.state() == -2 : broker injoignable (IP, port ou pare-feu)
}
```

Points clés :

- Compte dédié `esp` ; le mot de passe est dans `config.h`, hors Git.
- Combiné au TLS, le mot de passe circule **déjà chiffré** : impossible à capturer au sniffing.
- Le refus est diagnostiqué explicitement (état 5), ce qui sert de preuve que l'authentification est bien active.

Séparation code / secrets

Aucun secret n'est écrit en dur dans le sketch. Tous les secrets (SSID et mot de passe Wi-Fi, mot de passe MQTT, certificat de la CA) vivent dans `config.h`, exclu du dépôt Git. Le firmware refuse même de compiler si un secret manque, grâce à un garde-fou `#error`.

```
#if __has_include("config.h")
#include "config.h"
#else
#error "Il manque config.h dans le dossier du sketch."
#endif

#if !defined(WIFI_SSID) || !defined(WIFI_PASS) || !defined(MQTT_HOST) ||
!defined(MQTT_PASS)
#error "config.h doit définir WIFI_SSID, WIFI_PASS, MQTT_HOST et MQTT_PASS."
#endif
```

Points clés :

- Le code (`SentinelX.ino`) peut être versionné et partagé sans fuite de secret.

- `config.h` est listé dans `.gitignore` ; un modèle `config.example.h` (valeurs factices) documente les champs attendus.
- Le certificat de la CA est fourni à part (`ca_cert.h`), lui aussi hors Git.

Cloisonnement des topics

Le boîtier ne publie et ne lit que sur son propre préfixe, construit à partir de son identité :

```
#define TOPIC_BASE "sentinel/" GROUP_ID "/" DEVICE_ID //
sentinel/groupe1/edge01
// publication : .../telemetry .../alert .../status
// abonnement : .../cmd .../config
```

Ce préfixe correspond **exactement** à l'ACL du broker pour le compte `esp` : le boîtier ne peut écrire que SA télémétrie et ne lit que SES commandes. Ainsi, même si les identifiants du boîtier étaient compromis, un attaquant ne pourrait ni écrire sur les topics d'un autre boîtier, ni commander un autre équipement — le pouvoir du compte `esp` est limité à son seul périmètre (voir le document sécurité, faille F1).

Intégrité de présence — Last Will

À la connexion, le boîtier publie `online` (retained). Il déclare aussi un **Last Will** : si la connexion tombe (coupure, brouillage, sabotage), le broker publie automatiquement `offline` à sa place. Le serveur et le dashboard savent donc **immédiatement** qu'un boîtier a disparu — une tentative de neutralisation du capteur devient visible plutôt que de passer inaperçue.

```
mqtt.connect(DEVICE_HOSTNAME, MQTT_USER, MQTT_PASS,
             topicStatus, 1, true, "offline", true); // Last Will :
"offline" (retained)
mqtt.publish(topicStatus, "online", true);           // présence confirmée
à la connexion
```

Alarme silencieuse et empreinte mémoire

Par défaut, une intrusion détectée par le PIR est signalée au serveur **sans faire sonner le buzzer** (`buzzOnMotion = false`). L'alerte remonte au dashboard, mais le boîtier reste discret : l'intrus n'est pas averti qu'il a été repéré. Le buzzer n'est déclenché que sur commande explicite du dashboard, ou si l'opérateur active « Buzzer sur détection ».

```
bool armed          = DEFAULT_ARMED;           // détection d'intrusion active
bool buzzOnMotion = DEFAULT_MOTION_BUZZER;    // false : alarme silencieuse par
défaut

// à la détection PIR :
publishAlert("motion", m, armed ? "warning" : "info", NAN, NAN); // remonte
toujours
// le buzzer ne sonne que si buzzOnMotion est vrai, ou sur commande manuelle
```

Empreinte mémoire : avec le TLS actif (BearSSL + CA épinglée), le boîtier conserve environ **15,6 ko de heap libre** en fonctionnement — une marge suffisante et stable pour l'ESP8266. Le chiffrement n'empêche donc pas le fonctionnement normal du boîtier.